Neue Ausrichtung Pdvm System

Stand und Differenzen

Der derzeitige Entwicklungsstand ist so, dass die Globalisierung der System- und Datenpflege nicht konsequent umgesetzt werden konnte. Es gibt hier Dinge, die gemacht wurden, wie diese in bisheriger Technik so gehandhabt wurden und die als Rahmen beschriebenen Logik nicht detailliert dargestellt werden konnte.

Der Derzeitige Stand beruht auf den in den ARCHITECTURE_RULES beschriebenen Regeln die grundsätzlich weiter gelten.

Für das Pflegen von Daten und des Systems werden wir einen globalen Ansatz wählen, der linear und ausschließlich mit in der Datenbank beschriebenen Elementen funktioniert.

Tabellen

Die Namen der Tabellen werden grundsätzlich in kleinen Buchstaben geschrieben. Die ersten 3 Stellen bezeichnen die Anwendung, für die in der Tabelle die Daten sind. So beginnen alle Tabellen des System mit ‚sys' gefolgt von einem Unterstrich.

Die Struktur der Tabellen ist einheitlich. Die einzige Ausnahme hiervon ist die sys_benutzer, die zusätzlich die Spalten ‚benutzer' und ‚passwort' hat.

Alle Tabellen werden ausschließlich über PdvmDatabase gepflegt. Siehe auch ARCHITECTURE_RULES.

Allgemeine Zusammenhänge

Es ist so, dass es nur ganz wenige Ausnahmen von den allgemeinen Regeln gibt, die dann aber separat behandelt werden, um die allgemeinen Regeln linear und einfach zu halten. Dieses bedeutet, dass die einzelnen Schritte und Bereiche in einzelne Elemente gekapselt sind und die erforderlichen Informationen immer von der Datenbank kommen.

Die Keys für diese Elemente bzw. Bereiche sind immer GUIDs. Damit wird die Einmaligkeit und ein genauer Zugriff gewährleistet. Alle Daten sind im JSON Format in der Spalte ‚daten' in der einzelnen Tabelle.

Grundlegende Daten

Jeder Datensatz hat eine Gruppe = ROOT. Der Mindestinhalt sind die Eigenschaften:

- SELF_GUID → diese beinhaltete den Wert aus der Spalte „uid“. Dieser Wert kann nicht verändert werden, da es der PrimärKey des Datensatzes ist. Da die Datensatzstruktur überall gleich ist, können auch Sätze aus unterschiedlichen Tabellen gemischt werden. Diese macht später in der APP Entwicklung ggf. sinn.
- SELF_NAME → dieses ist der Wert aus der Spalte „name“. Dieser kann geändert werden und ist mit jedem Speichervorgang zu synchronisieren. Hier führen wir Namenskonventionen ein. Die ersten 3 Zeichen sind in Grossbuchstaben die Applikation (wie die ersten 3 Zeichen der Tabellen) gefolgt von einem Leerzeichen.

Diese Felder müssen in jeder Tabelle im Satz der GUID = 66666666-6666-6666-6666-666666666666 mindestens enthalten sein. Anderenfalls ist bei der Neuanlage ein Fehler auszugeben.

Neuer Satz in einer Tabelle

Ein neuer Satz in einer Tabelle wird im Dialog (handler: go_dialog) durchgeführt auf die Tabelle in „sys_dialogdaten ROOT.TABLE“ und es wird der entsprechende Button nur angeboten, wenn „sys_dialogdaten ROOT.GO_NEW_SET“ = true ist. Der Button ist auf dem Tab mit dem MODULE=view darzustellen

Damit ein Neuer Satz angelegt werden kann müssen Sätze mit den fiktiven GUIDs 66666666-6666-6666-6666-666666666666 und 55555555-5555-5555-5555-555555555555 vorhanden sein. Ist dieses nicht der Fall, dann einen Fehlermeldung und kein neuer Satz.

Ablauf:

- Name wird abgefragt und auf den Präfix abgeprüft (Tabelle ist ja bekannt)
- Inhalt von 66666666-6666-6666-6666-666666666666.daten wird die Basisstruktur des neuen Satzes. Hier werden
    - SELF_GUID und SELF_NAME gesetzt.
- Sind weitere Gruppen in der bisherigen Struktur vorhanden, so werden diese in 55555555-5555-5555-5555-555555555555 unter der Gruppe „TEMPLATES“ gesucht und die entsprechende Struktur eingefügt.
  Im Prinzip wird dieses bei jeder leeren Eigenschaft gemacht, wenn diese in sys_control_dict den type ‚element_list‘ hat.
- Neuer Satz wird gespeichert und zum Editieren angeboten.

Dialog

Der Dialog wird mit dem Handler „go_dialog“ und der dialog_guid aufgerufen. Der Dialog managt die Module in den parametrisierten Tabs. Die Module selbst sind grundsätzlich gekapselt und beruhen auf eigenen Beschreibungen. Als Beispiel eine typische Dialogstruktur:

```
{
  "ROOT": {
    "TABS": 2,
    "TABLE": "sys_control_dict",
    "SELF_GUID": "9f06711e-4ad8-4ea4-9837-2f40f3a6f101",
    "SELF_NAME": "sys_control_dict Dialog",
    "DIALOG_TYPE": "norm",
    "GO_NEW_SET": true,
    "TAB_ELEMENTS": {
      "2bba7fbe-74d1-4622-abba-9763d30b3166": {
        "TAB": 1,
        "GUID": "9f06711e-4ad8-4ea4-9837-2f40f3a6f102",
        "HEAD": "Liste",
        "TABLE": "sys_viewdaten",
        "MODULE": "view",
        "EDIT_TYPE": "",
        "OPEN_EDIT": "double_click",
        "EDIT_TAB": 2,
        "SELECTION_MODE": "single"
      },
      "28a9fcf6-27f8-40ed-81bd-a03e97921f23": {
        "TAB": 2,
        "GUID": "9f06711e-4ad8-4ea4-9837-2f40f3a6f103",
        "HEAD": "Bearbeiten",
        "TABLE": "sys_framedaten",
        "MODULE": "edit",
        "EDIT_TYPE": "pdvm_edit",
        "OPEN_EDIT": "",
        "EDIT_TAB": 0,
        "SELECTION_MODE": ""
      }
    }
  }
}
```


Hier sind alle Daten unter der Gruppe ROOT. Die TAB_ELEMENTS ist eine element_list (in sys_control_dict).

Die GUID in den Tabs bezieht sich immer auf die TABLE in den Tabs. Also Tab 1 holt sich für das Module = view die Daten dazu aus der sys_viewdaten mit der uid = 9f06711e-4ad8-4ea4-9837-2f40f3a6f102 und der Tab 2 holt sich für das Module= pdvm_edit die Daten aus sys_framdaten mit der uid = 9f06711e-4ad8-4ea4-9837-2f40f3a6f103. Diese Vorgehensweise ermöglicht es ohne Änderung Module einzufügen deren Definitionen in anderen Tabellen liegen. Dieses ist sinnvoll bei der späteren App Entwicklung.

Dictionary – Controls

Es ist eine Art Dictionary für Properties und Felder vorhanden. In diesem sind die Controls / Eigenschaften vorhanden. Es geht hier darum nach Möglichkeit eine Definition nur an einer definierten Stelle zu haben. Dabei gibt es verschiedene Regeln:

1. Die Eigenschafen der Tabellen, die mit „sys_“ beginnen werden in der sys_control_dict abgelegt. Diese befindet sich in der Systemdatenbank (pdvm-system).
2. Die Eigenschaften der Tabellen mit Ausnahme der Ziffer 1 werden in der sys_contr_dict_man abgelegt. Diese befindet sich in der Mandantendatenbank.
3. Jedes dieser Controls hat einen eigenen Datensatz. Key = uid
4. Der name (SELF_NAME) hat folgende Konvention: Er besteht aus „ [Präfix der Tabelle] + [_] + Property/Feld“ in Großbuchstaben
5. Die Bildung eines Satzes erfolgt nach den definierten Regeln über die fiktiven GUIDS (aus 6en und 5en)

Hier das Muster eines solchen Satzes:

```
{
  "ROOT": {
    "SELF_GUID": "",
    "SELF_NAME": ""
  },
  "CONTROL": {
      "NAME": "",
      "TYPE": "",
      "FIELD": "",
      "LABEL": "",
      "TABLE": "",
      "GRUPPE": "",
      "ABDATUM": false,
      "DEFAULT": "",
      "UPPER_CASE": false,
      "SORTABLE": true,
      "READ_ONLY": false,
      "HISTORICAL": false,
      "SEARCHABLE": true,
      "EXPERT_MODE": true,
      "FILTER_TYPE": "contains",
      "PARENT_GUID": null,
      "SOURCE_PATH": "root",
      "DISPLAY_SHOW": true,
      "EXPERT_ORDER": 0,
      "DISPLAY_ORDER": 0,
      "SORT_DIRECTION": "asc",
      "FIELDS_ELEMENTS": {},
      "CONFIGS_ELEMENTS": {},
```

```
        "SORT_BY_ORIGINAL": false
        }
}
```

Wie ersichtlich ist, sind darin alle Werte eines Controls, egal ob frame oder view enthalten. Damit erfolgen der Aufbau der Controls in View und Frame über die uid des Controls. Einzelne Properties können dabei überschrieben werden. Mit der Verwendung der Controls werden diese intern unter dem Namen aufgelöst, verwendet und in der Systemsteuerung persistent gemacht. Mit einem neuen Aufruf werden diese abgeglichen, dieses kann auch über die Gestaltung eines Änderungsmechanismus erfolgen.

Damit es verständlich ist eine Tabelle mit den Properties/Controls aus den bisherigen Strukturen.

SELF_NAME	***NAME***	***TYPE***	***UPPER_CASE***	***Anmerkung***
SYS_SELF_NAME	SELF_NAME	string	true	
SYS_NAME	NAME	string	false	
SYS_TYPE	TYPE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_UPPER_CASE	UPPER_CASE	true_false	false	
SYS_ROOT	ROOT	element_list	false	Immer read_only. Benötigt Eintrag in FIELDS_ELEMENTS der aber in der Frame erfolgt
SYS_SELF_GUID	SELF_GUID	string	false	immer read_only
SYS_FIELD	FIELD	dropdown	true	Benötigt Eintrag in CONFIG_ELEMENTS suche in sys_control_dict
SYS_LABEL	LABEL	string	false	
SYS_TABLE	TABLE	pre_string	true	
SYS_GRUPPE	GRUPPE	string	true	
SYS_DEFAULT	DEFAULT	string	false	
SYS_SORTABLE	SORTABLE	true_false	false	
SYS_READ_ONLY	READ_ONLY	true_false	false	
SYS_HISTORICAL	HISTORICAL	true_false	false	
SYS_SEARCHABLE	SEARCHABLE	true_false	false	
SYS_EXPERT_MODE	EXPERT_MODE	true_false	false	
SYS_FILTER_TYPE	FILTER_TYPE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_PARENT_GUID	PARENT_GUID	string	false	wird in der Regel maschinell gesetzt
SYS_SOURCE_PATH	SOURCE_PATH	string	false	
SYS_DISPLAY_SHOW	DISPLAY_SHOW	true_false	false	
SYS_DISPLAY_ORDER	DISPLAY_ORDER	number	false	
SYS_EXPERT_ORDER	EXPERT_ORDER	number	false	
SYS_SORT_DIRECTION	SORT_DIRECTION	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS

SYS_FIELDS_ELEMENTS	FIELD_ELEMENTS	element_list	false	Benötigt Eintrag in TEMPLATE (uid= 5er) oder frame_guid
SYS_CONFIGS_ELEMENTS	CONFIGS_ELEMENTS	element_list	false	Benötigt Eintrag in TEMPLATE (uid= 5er) oder frame_guid
SYS_TABS	TABS	number	false	
SYS_EDIT_TYPE	EDIT_TYPE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_DIALOG_TYPE	DIALOG_TYPE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_GO_NEW_SET	GO_NE_SET	true_false	false	
SYS_TAB_ELEMENTS	TAB_ELEMENTS	element_list	false	Benötigt Eintrag in TEMPLATE (uid= 5er) oder frame_guid
SYS_TAB	TAB	number	false	
SYS_GUID	GUID	string	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_MODULE	MODULE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_OPEN_EDIT	OPEN_EDIT	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_EDIT_TAB	EDIT_TAB	number	false	
SYS_SELECTION_MODE	SELECTION_MODE	dropdown	false	Benötigt Eintrag in CONFIGS_ELEMENTS
SYS_CONTROL	CONTROL	element_list	false	Immer read_only. Benötigt Eintrag in FIELDS_ELEMENTS der aber in der Frame erfolgt

View

Die View wird in der heutigen Funktionalität erhalten. Mit der Umstellung wird die View funktionsfähig erhalten. Sobald der EDIT_TYPE = pdvm_edit so weit fertig ist, wird die View nochmals überarbeitet.

EDIT – Type: pdvm_edit

Der PdvmEdit ist ein Edit im Rahmen eines Dialogs, der in den Framedaten nach einem flachen Muster beschrieben wird .

Es wird neu gestaltet. Grundzüge sind in der alten DesktopApplikation ersichtlich. Hier steht das meiste in den Dateien pdvm_input_control.py und pdvm_input_control_manager.py.

Der grundlegende Aufbau sieht so aus. Es gibt die Gruppe = ROOT – hier werden auch die Tabs als element-list definiert. Ansonsten gibt es für die entsprechende Tabelle eine Gruppe (Name der Tabelle in Großbuchstaben). Darunter liegen die Felder (einzelne Controls für ein Feld – InputControl (kann auch eine element_list sein).

In den Tabs werden diese Felder (Controls) nach den Tab und der display_order bzw. expert_order angezeigt.

Die grundlegende Beschreibung eines Inputcontrol steht im entsprechenden Datensatz in sys_control_dict … die in den FrameDaten überschrieben und ergänzt werden können.

Es ist klar, dass die Frames neu aufgebaut werden müssen.

Für die Darstellung und Steuerungen ist der TYPE zuständig:

TYPE	***Erläuterung***
json	Dieser Type wird verwendet, wenn kein Eintrag in im dict. Vorhanden ist. Es wird der JSON dargestellt, der im bezeichneten Item liegt.
string	Es wird der String des Items dargestellt
text	Es wird ein mehrzeiliger Text in HTML dargestellt bzw. zum Editieren angeboten, sofern nicht read_only ist.
true_false	Es wird ein Checkbutton angeboten
date	Es wird das Datum mit dem PdvmDateTimePicker dargestellt bzw. zum Editieren angeboten
time	Es wird die Zeit mit dem PdvmDateTimePicker dargestellt bzw. zum Editieren angeboten
datetime	Es wird das Datum und die Zeit mit dem PdvmDateTimePicker dargestellt bzw. zum Editieren angeboten.
dropdown	benötigt einen Eintrag in CONFIGS_ELEMENTS mit dem namen ‚dropdown' … die unterschiedlichen Modi sind nachfolgend beschrieben.

Dropdown – Erläuterung

In einer Eigenschaft mit dem Type ‚dropdown' ist unter den CONFIGS_ELEMENTS ein Block mit dem Namen ‚dropdown' erforderlich:

```
"dropdown": {
  "key": "",
  "field": "",
  "table": "",
  "group": ""
}
```

Hierbei haben verschiedene Konstellation verschieden Bereitstellungen der Auswahldaten:

key	***field***	***table***	***group***	***Erläuterung***
Guid	Feld	Tabelle	""	in der angegebenen Tabelle wird unter der Gruppe = Sprache in Großbuchstaben (DE-DE) das Feld gesucht und die darin enthaltene Auswahl angeboten
Guid	Feld	Tabelle	Gruppe	in der angegebenen Tabelle wird unter der Gruppe das Feld

				gesucht und die Auswahl angeboten.
view_guid	“”	“”	*VIEW	Hier wird mit den Angaben aus der view_guid eine View zur Auswahl angeboten und die uid übernommen
view_guid	“”	Tabelle	*VIEW	Hier wird mit den Angaben aus der view_guid und der angegebenen Tabelle eine View zur Auswahl angeboten und die uid übernommen

Anmerkung: *der key eines Feldes ist die uid aus sys_control_dict. Wird folglich über einen Auswahl übernommen. Genauso verhält es sich mit der group.*

Hilfe – Erläuterung

In allen Eigenschaften kann unter den CONFIGS_ELEMENTS ein Block mit dem Namen ‚help‘ vorhanden sein:

```
"help": {
  "key": "",
  "field": "",
  "table": "",
  "group": ""
},
```

Dieselbe Beschreibung gilt hier auch für ‚tooltip‘. Diese werden auch sprachbezogen abgelegt.

key	***field***	***table***	***group***	***Erläuterung***
Guid	Feld	Tabelle	“”	in der angegebenen Tabelle wird unter der Gruppe = Sprache in Großbuchstaben (DE-DE) das Feld gesucht und Text ausgegeben
Guid	Feld	Tabelle	Gruppe	in der angegebenen Tabelle wird unter der Gruppe das Feld gesucht und der Text ausgegeben

Anmerkung: *Die Speicherung und Ausgabe erfolgt hier im HTML Format, damit eine gut formatierte Darstellung erfolgen kann. Ist ein HTML Format nicht gespeichert, so erfolgt die Ausgabe als Standardtext.*

Sonstige Dinge für den pdvm_edit

Da das pdvm_inputcontrol (picontrol) ein in sich geschlossener Baustein ist benötigt er auf Dauer weitere Funktionalitäten, die von vornherein mit angedachte werden müssen.

- Hilfebutton (icon) → holt die Informationen aus CONTROLS_ELEMENTS. Sofern keine Informationen vorliegen, ist der Button nicht aktiv.
- Historiebutton (icon) → ist nur aktiv, wenn HISTORICAL = true ist. Die Funktionalität wird zu einem anderen Zeitpunkt hinzugefügt.
- Änderungsnachweis (icon) → die Funktionalität wird später erstelle, daher ist der Button noch nicht aktiv.

Zu diesen und ggf. auch andere Buttons sollten wir Tooltips hinterlegen können. Nich im Sources, sondern in der Datenbank. Ich stelle es mir so vor, dass wir in sys_control_dict für die Button Sätze hinterlegen in denen man dann zum Button in CONTROLS_ELEMENTS Hilfetexte und Tooltips verlinken kann. Alles nach einem Schema. Dieses ginge zum Beispiel mit einen Präfix der nicht dem TabellenPräfix entspricht sondern ein geschützter Präfix für die Buttons und andere Elemente verwendet wird. Hier könnt ich mir gut XXX vorstellen. Bitte näher betrachten, ob dieses eine angemessen lineare Lösung ist oder ob es etwas Optimaleres gibt.

Idee im Rahmen der Darstellung

Wenn man einen großen Bildschirm hat, mit dem wir gerade entwickeln, dann macht es beim pdvm_edit folgende Dinge sinn:

- Begrenzung der Breite des Inputcontrols, damit die rechts angebrachten Buttons nicht so weit weg sind
- Es könnten immer zwei Tabs nebeneinander im Content dargestellt und verwaltet werden. Die Daten eines Editdialoges sind ja über entsprechende Instanzen immer im Zugriff.

Prüfung und weiteres Vorgehen

Das System ist so aufgebaut, dass alles, so weit möglich, linear in den Daten in den unterschiedlichen Datenbanken liegen. Hier sind auch in Zukunft die ARCHITECTURE_RULES zu beachten. Sollte es hier irgendwo Schwierigkeiten geben müssen wir uns darüber direkt unterhalten, ob wir diese Regeln ändern können.

Ich schlage folgende Schritte vor:

1. Prüfe die Logik und die vorgeschlagenen Strukturen, ob die so Möglich sind, oder noch optimiert werden können.
2. Beschreibe die neuen Strukturen und zeige auf was zwingend geändert werden muss.
3. Wir passen alles an, damit die VIEW mit dem Anlegen neuer Sätze funktioniert. Ich denke wir löschen dazu in sys_control_dict alle Datensätze bis auf die mit den fiktiven guids aus den 0ern, 5ern und 6ern. Also bleiben 3 Datensätze stehen.
4. Die zu Beginn erforderlichen Controls werde ich manuell mit Hilfe von edit_json anlegen.
5. Neuerstellung von pdvm_edit. Die dazu erforderlichen Framedaten werde ich manuell erstellen. Ohne Framedaten muss er einfach einen leeren Content darstellen. Ich möchte die Framedaten dann Zug um Zug aufbauen, um zu sehen, wie es funktioniert. Damit kann ich den Ablauf auch gleich von vornherein korrigieren.
6. Wenn andere Dinge zum Ablauf oder der Logik sinnvoller sind, zeige es mir auf. Es sollte alles linear sein und alles, auch schon zur Entwicklung, entsprechende auf den Datenbanken eingestellt werden. Der Source muss aus den Daten dann nur die Anwendung machen.